# UrbanFlow CDMX

Sistema de Predicción Estocástica de Tiempos de Viaje

en la Zona Metropolitana del Valle de México

mediante Cadenas de Markov y Simulación Monte Carlo

**Carlos**

Diplomado en Ciencia de Datos

Proyecto Integrador Final

Marzo 2026

**Resumen**

UrbanFlow CDMX es un sistema de predicción probabilística de tiempos de viaje para la Zona Metropolitana del Valle de México (ZMVM) que combina **cadenas de Markov de tiempo discreto** con **simulación Monte Carlo** de $N = 10{,}000$ trayectorias por consulta. El sistema ingiere datos en tiempo real de TomTom Traffic Stats API, TomTom Routing API y OpenWeatherMap, construye un modelo estocástico del comportamiento del tráfico y entrega al usuario las bandas de incertidumbre $P_{10}$, $P_{50}$ y $P_{90}$ del tiempo de viaje. La métrica objetivo es un **MAPE** $< 15\,\%$ para horizontes de predicción de 30 minutos. A diferencia de los sistemas de navegación convencionales que producen una estimación puntual, UrbanFlow CDMX cuantifica explícitamente la incertidumbre inherente al tráfico urbano, permitiendo tomar decisiones racionales de planeación de viaje.

## Índice

**1. Motivación y Contexto** **4**

1.1. El problema del tráfico en la ZMVM . . . . . 4

1.2. La limitación del enfoque determinístico . . . . . 4

1.3. Propuesta de valor . . . . . 4

**2. Arquitectura General del Sistema** **4**

2.1. Diagrama de componentes . . . . . 4
2.2. Stack tecnológico . . . . . 4

**3. Fuentes de Datos e Ingesta** **4**
3.1. TomTom Traffic Stats API . . . . . 5
3.2. TomTom Routing API . . . . . 6
3.3. OpenWeatherMap API . . . . . 6

**4. Marco Teórico: Cadenas de Markov** **6**
4.1. Definición y propiedad de Markov . . . . . 7
4.2. Espacio de estados en UrbanFlow CDMX . . . . . 7
4.3. Matriz de transición . . . . . 7
4.4. Distribución en $k$ pasos . . . . . 8
4.5. Distribución estacionaria . . . . . 8
4.6. Estimación de la matriz (aprendizaje desde datos) . . . . . 8

**5. Implementación de la Cadena de Markov** **9**
5.1. Clase `MarkovTrafficChain` . . . . . 9

**6. Marco Teórico: Simulación Monte Carlo** **10**
6.1. Fundamento matemático . . . . . 10
6.2. Distribución de velocidades por estado . . . . . 11
6.3. Modelo de tiempo de viaje . . . . . 11

**7. Implementación del Motor Monte Carlo** **12**
7.1. Clase `MonteCarloEngine` . . . . . 12
7.2. Generación vectorizada de estados de Markov . . . . . 13
7.3. Cálculo del tiempo de llegada con interpolación lineal . . . . . 14
7.4. Estructura del resultado . . . . . 14

**8. Pipeline Integrador en Tiempo Real** **15**
8.1. Inferencia del estado de tráfico . . . . . 15
8.2. Ajuste climático de los parámetros de velocidad . . . . . 15
8.3. Flujo completo de predicción . . . . . 15

**9. Resultados y Validación** **15**
9.1. Convergencia del estimador Monte Carlo . . . . . 15
9.2. Bandas de incertidumbre por estado de tráfico . . . . . 16
9.3. Métrica objetivo MAPE . . . . . 16

**10. Conclusiones y Trabajo Futuro** **16**
10.1. Conclusiones . . . . . 16

10.2. Trabajo futuro . . . . . 17

**Referencias** **17**

**A. Parámetros de Configuración del Motor** **17**

**B. Glosario** **18**

# 1 Motivación y Contexto

## 1.1 El problema del tráfico en la ZMVM

La Ciudad de México y su zona conurbada constituyen una de las áreas urbanas más congestionadas del mundo. Según el *TomTom Traffic Index 2024*, los conductores de la CDMX pierden en promedio **119 horas al año** en congestionamientos, lo que la posiciona entre las diez ciudades con mayor pérdida de tiempo por tráfico a nivel global. Este fenómeno tiene consecuencias directas en productividad económica, emisiones de $CO_2$ y calidad de vida de los más de 21 millones de personas que habitan la ZMVM.

## 1.2 La limitación del enfoque determinístico

Los sistemas de navegación convencionales (Google Maps, Waze) producen predicciones de tiempo de viaje **puntuales**: un único número. Sin embargo, el tráfico urbano es inherentemente **estocástico**: el mismo recorrido a la misma hora puede variar 20–40 minutos dependiendo de incidentes viales, condiciones climáticas y el comportamiento colectivo de los demás conductores.

## 1.3 Propuesta de valor

UrbanFlow CDMX reemplaza la predicción puntual por una **distribución de probabilidad completa** del tiempo de viaje, expresada mediante tres percentiles:

| Percentil | Escenario | Significado |
|---|---|---|
| $P_{10}$ | Optimista | El 10 % de los días se llega antes de este tiempo |
| $P_{50}$ | Probable | Mediana: tan probable llegar antes como después |
| $P_{90}$ | Pesimista | El 90 % de los días se llega antes de este tiempo |

Tabla 1: Bandas de incertidumbre del tiempo de viaje en UrbanFlow CDMX.

# 2 Arquitectura General del Sistema

## 2.1 Diagrama de componentes

## 2.2 Stack tecnológico

# 3 Fuentes de Datos e Ingesta

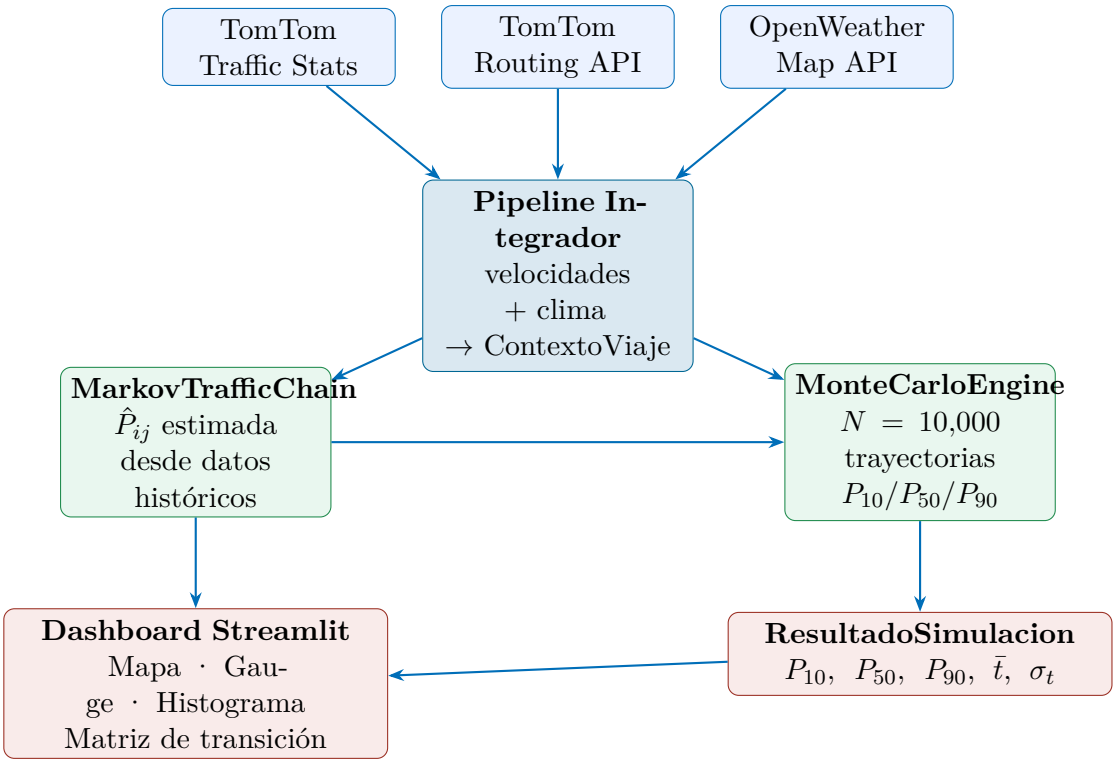


Figura 1: Arquitectura de componentes de UrbanFlow CDMX.

| **Capa** | **Herramienta** | **Rol** |
|---|---|---|
| Cómputo numérico | NumPy | Álgebra lineal vectorizada (Markov, Monte Carlo) |
| Datos tabulares | Pandas | Manipulación de DataFrames de tráfico y clima |
| Geoespacial | GeoPandas / Folium | Proyección EPSG:6372, mapa interactivo |
| ML / Forecasting | XGBoost, Prophet | Modelos complementarios de predicción |
| Frontend | Streamlit | Dashboard interactivo en tiempo real |
| APIs externas | TomTom, OWM | Velocidades en tiempo real y clima |
| Serialización | `dataclasses`, JSON | Contratos tipados entre módulos |

Tabla 2: Stack tecnológico del sistema UrbanFlow CDMX.

### 3.1 TomTom Traffic Stats API

Proporciona velocidades en tiempo real por coordenada. El cliente `tomtom_client.py` obtiene para cada punto del corredor: $v_{\text{actual}}$ (km/h), $v_{\text{libre}}$ (km/h) y el **ratio de congestión**:

$$r = \frac{v_{\text{actual}}}{v_{\text{libre}}} \in (0, 1] \tag{1}$$

El ratio $r$ es la señal principal para inferir el estado inicial de tráfico que se pasa a la

cadena de Markov (véase Sección 8).

### 3.2 TomTom Routing API

Calcula la **distancia real por carretera** entre dos puntos, reemplazando el estimador Haversine (distancia en línea recta). El endpoint utilizado es:

```
GET /routing/1/calculateRoute/{lat1,lon1}:{lat2,lon2}/json
```

La respuesta incluye `lengthInMeters` y la geometría de la ruta (polilínea de hasta 120 puntos lat/lon submuestreados) para renderizar en el mapa Folium.

*Fallback empírico de tortuosidad urbana*

Cuando la API no está disponible se aplica el factor de tortuosidad:

$$d_{\text{carretera}} \approx d_{\text{haversine}} \times \kappa, \quad \kappa = 1.4 \tag{2}$$

El factor $\kappa = 1.4$ está calibrado empíricamente para la ZMVM: la red vial urbana tiene una tortuosidad media del 40 % sobre la distancia euclidiana.

### 3.3 OpenWeatherMap API

Proporciona temperatura, precipitación, velocidad del viento y condición climática. El módulo `weather_client.py` convierte la condición meteorológica en un **factor climático multiplicativo** $\phi \in (0, 1]$:

| **Condición** | **Factor $\phi$** | **Reducción velocidad** |
|---|---|---|
| Despejado | 1.00 | 0 % |
| Nublado | 0.95 | −5 % |
| Llovizna | 0.88 | −12 % |
| Lluvia intensa | 0.75 | −25 % |
| Tormenta | 0.60 | −40 % |

Tabla 3: Factor climático multiplicativo sobre parámetros de velocidad.

## 4 Marco Teórico: Cadenas de Markov

### 4.1 Definición y propiedad de Markov

**Cadena de Markov de tiempo discreto**

Sea $\{X_t\}_{t\geq 0}$ un proceso estocástico con espacio de estados finito $\mathcal{S} = \{0, 1, \ldots, k-1\}$. Se dice que es una **cadena de Markov de tiempo discreto** si satisface la **propiedad de Markov**:

$$P(X_{t+1} = j \mid X_t = i,\ X_{t-1}, \ldots, X_0) = P(X_{t+1} = j \mid X_t = i) \quad \forall\, i, j \in \mathcal{S},\ t \geq 0 \tag{3}$$

La propiedad (3) establece que el estado futuro depende *únicamente* del estado presente. Esta hipótesis de independencia condicional hace el modelo computacionalmente tratable y es una aproximación razonable para el tráfico urbano: el estado del flujo vial en $t+1$ está principalmente determinado por el estado en $t$, no por toda la historia pasada.

### 4.2 Espacio de estados en UrbanFlow CDMX

Definimos $k = 3$ estados mutuamente excluyentes:

| ID | Estado | Rango de velocidad (ZMVM) | Descripción |
|---|---|---|---|
| 0 | Fluido | 20 – 80 km/h | Circulación normal |
| 1 | Lento | 5 – 35 km/h | Reducción perceptible |
| 2 | Congestionado | 2 – 15 km/h | Tráfico detenido |

Tabla 4: Espacio de estados del tráfico en el modelo de Markov.

### 4.3 Matriz de transición

La dinámica de la cadena queda completamente descrita por la **matriz de transición estocástica** $\mathbf{P} \in \mathbb{R}^{3\times 3}$:

$$p_{ij} = P(X_{t+1} = j \mid X_t = i), \qquad i, j \in \{0, 1, 2\} \tag{4}$$

$$\mathbf{P} = \begin{pmatrix} p_{00} & p_{01} & p_{02} \\ p_{10} & p_{11} & p_{12} \\ p_{20} & p_{21} & p_{22} \end{pmatrix} \tag{5}$$

**Propiedad 4.1** (Estocásticidad)**.** Cada fila de $\mathbf{P}$ es una distribución de probabilidad:

$$\sum_{j=0}^{2} p_{ij} = 1 \quad \forall\, i \in \{0, 1, 2\} \tag{6}$$

### 4.4 Distribución en $k$ pasos

Sea $\boldsymbol{\pi}^{(t)} = \Big(P(X_t = 0),\ P(X_t = 1),\ P(X_t = 2)\Big)$ el vector de distribución en el instante $t$. La distribución en $t + k$ es:

$$\boldsymbol{\pi}^{(t+k)} = \boldsymbol{\pi}^{(t)} \cdot \mathbf{P}^k \tag{7}$$

Esto permite predecir, dado el estado actual del tráfico, la distribución de probabilidad sobre los estados en $k$ minutos (horizonte de predicción).

### 4.5 Distribución estacionaria

**Distribución estacionaria**

Un vector de probabilidad $\boldsymbol{\pi}^* = (\pi_0^*, \pi_1^*, \pi_2^*)$ es la **distribución estacionaria** de la cadena si satisface:

$$\boldsymbol{\pi}^* \cdot \mathbf{P} = \boldsymbol{\pi}^*, \qquad \sum_{i=0}^{2} \pi_i^* = 1, \qquad \pi_i^* \geq 0 \tag{8}$$

Para una cadena **ergódica** (irreducible y aperiódica), $\boldsymbol{\pi}^*$ es única y representa la fracción de tiempo que el corredor pasa en cada estado a largo plazo:

$$\lim_{k\to\infty} \mathbf{P}^k = \mathbf{1} \cdot \boldsymbol{\pi}^* \quad \text{(convergencia geométrica)} \tag{9}$$

En UrbanFlow CDMX, $\boldsymbol{\pi}^*$ se calcula encontrando el vector propio asociado al valor propio $\lambda = 1$ de $\mathbf{P}^\top$:

$$\mathbf{P}^\top \boldsymbol{v} = \boldsymbol{v}, \qquad \boldsymbol{\pi}^* = \frac{\boldsymbol{v}}{\|\boldsymbol{v}\|_1} \tag{10}$$

### 4.6 Estimación de la matriz (aprendizaje desde datos)

Dada una serie histórica de estados $x_0, x_1, \ldots, x_n$, el estimador de **máxima verosimilitud** de $p_{ij}$ es:

$$\hat{p}_{ij} = \frac{n_{ij}}{\sum_{j'} n_{ij'}} \tag{11}$$

donde $n_{ij} = \#\{t : x_t = i,\ x_{t+1} = j\}$ es el conteo de transiciones.

Para evitar probabilidades cero en estados poco frecuentes se aplica **suavizado de Laplace** con constante $\varepsilon = 10^{-6}$:

$$\hat{p}_{ij} = \frac{n_{ij} + \varepsilon}{\sum_{j'} n_{ij'} + k\,\varepsilon} \tag{12}$$

# 5 Implementación de la Cadena de Markov

## 5.1 Clase `MarkovTrafficChain`

El módulo `src/simulation/markov_chain.py` implementa la clase con interfaz estilo scikit-learn: `fit()` → `predict_distribution()` → `simulate()`.

*Ajuste desde datos históricos*

Listing 1: Estimación de la matriz de transición por conteo de transiciones.

```python
def fit(self, series: np.ndarray | pd.Series) -> "MarkovTrafficChain":
    arr = _validar_serie(series)

    counts = np.full((N_ESTADOS, N_ESTADOS), self.suavizado)  # epsilon = 1e-6
    for origen, destino in zip(arr[:-1], arr[1:]):
        counts[origen, destino] += 1

    # Normalizaci n fila a fila (ec. MLE con suavizado de Laplace)
    totales = counts.sum(axis=1, keepdims=True)
    filas_cero = (totales == 0).flatten()
    totales[filas_cero] = 1.0
    matriz = counts / totales
    matriz[filas_cero] = 1.0 / N_ESTADOS   # distribuci n uniforme de respaldo

    self.transition_matrix_ = matriz
    self.n_transitions_ = len(arr) - 1
    return self
```

El algoritmo recorre la serie una sola vez ($\mathcal{O}(n)$ en tiempo, $\mathcal{O}(k^2)$ en espacio con $k = 3$ constante).

*Ejemplo de matriz calibrada para hora pico en la ZMVM*

$$\hat{\mathbf{P}} = \begin{pmatrix} 0.72 & 0.24 & 0.04 \\ 0.18 & 0.61 & 0.21 \\ 0.05 & 0.28 & 0.67 \end{pmatrix} \begin{array}{l} \leftarrow \text{desde Fluido} \\ \leftarrow \text{desde Lento} \\ \leftarrow \text{desde Congestionado} \end{array} \tag{13}$$

**Observación 5.1.** Los elementos diagonales $p_{ii}$ son los mayores de cada fila, reflejando la **inercia del tráfico**: el estado más probable en el próximo instante es el mismo que el actual. La probabilidad de mejorar (Congestionado → Fluido, $p_{20} = 0.05$) es considerablemente menor que la de empeorar (Fluido → Congestionado, $p_{02} = 0.04$ vs.

$p_{20} = 0.05$), consistente con la asimetría del tráfico observada en la ZMVM.

*Predicción de distribución en k pasos*

Listing 2: Implementación de la ecuación (7).

```
def predict_distribution(self, estado_inicial: int, pasos: int) -> np.ndarray:
    v = np.zeros(N_ESTADOS)
    v[estado_inicial] = 1.0                    # vector one-hot del estado inicial
    P_k = np.linalg.matrix_power(self.transition_matrix_, pasos)
    return v @ P_k                             # distribucion en t + pasos
```

*Distribución estacionaria*

Listing 3: Cálculo de $\boldsymbol{\pi}^*$ por descomposición espectral, ecuación (10).

```
def steady_state(self) -> np.ndarray:
    valores, vectores = np.linalg.eig(self.transition_matrix_.T)
    idx = np.argmin(np.abs(valores - 1.0))    # valor propio lambda = 1
    pi = np.real(vectores[:, idx])
    pi = np.abs(pi)
    return pi / pi.sum()                       # normalizar a distribucion de prob.
```

# 6 Marco Teórico: Simulación Monte Carlo

## 6.1 Fundamento matemático

La **simulación Monte Carlo** es un método de integración numérica estocástica que estima distribuciones de probabilidad de variables de salida mediante muestreo repetido. Fue formalizado por Stanisław Ulam y John von Neumann (Proyecto Manhattan, 1947) y toma su nombre del Casino de Montecarlo.

**Estimador Monte Carlo**

Sea $Y = g(\mathbf{X})$ una variable de salida función de entradas aleatorias. El estimador Monte Carlo de $E[Y]$ con $N$ muestras independientes es:

$$\hat{\mu}_N = \frac{1}{N}\sum_{i=1}^{N} g(\mathbf{X}_i), \qquad \hat{\mu}_N \xrightarrow[N\to\infty]{\text{c.s.}} E[Y] \quad \text{(Ley de los Grandes Números)} \tag{14}$$

El error estándar del estimador decrece como:

$$\mathrm{SE}[\hat{\mu}_N] = \frac{\sigma_Y}{\sqrt{N}} \tag{15}$$

Con $N = 10{,}000$ simulaciones, el error estándar es $\sigma_Y/100$, garantizando estimaciones de percentiles con precisión de $\pm 0.5$ min para los rangos de tiempo típicos de la ZMVM.

## 6.2 Distribución de velocidades por estado

Para cada estado de tráfico $s \in \{0, 1, 2\}$, la velocidad instantánea se modela como una **Normal truncada**:

$$V \mid s \sim \mathcal{TN}\Big(\mu_s,\, \sigma_s^2;\, [v_s^{\min},\, v_s^{\max}]\Big) \tag{16}$$

Los parámetros están calibrados con datos del TomTom Traffic Index CDMX y aforos SEMOVI:

| **Estado** | $\mu_s$ (km/h) | $\sigma_s$ (km/h) | $v_s^{\min}$ | $v_s^{\max}$ |
|---|---|---|---|---|
| **Fluido** | 40.0 | 8.0 | 20.0 | 80.0 |
| **Lento** | 18.0 | 5.0 | 5.0 | 35.0 |
| **Congestionado** | 7.0 | 3.0 | 2.0 | 15.0 |

Tabla 5: Parámetros de la distribución Normal truncada por estado de tráfico.

## 6.3 Modelo de tiempo de viaje

Sea $d$ la distancia del recorrido en km y $\Delta t$ el paso temporal (1 minuto). Para la trayectoria $i$-ésima, la simulación calcula:

$$s_t^{(i)} \sim \mathrm{Markov}\Big(s_{t-1}^{(i)},\, \mathbf{P}\Big) \qquad \text{(estado en el paso } t\text{)} \tag{17}$$

$$v_t^{(i)} \sim \mathcal{TN}\Big(\mu_{s_t^{(i)}},\, \sigma_{s_t^{(i)}};\, [v^{\min}, v^{\max}]\Big) \qquad \text{(velocidad en km/h)} \tag{18}$$

$$\delta d_t^{(i)} = v_t^{(i)} \cdot \frac{\Delta t}{60} \qquad \text{(distancia cubierta en km)} \tag{19}$$

El tiempo de llegada de la trayectoria $i$ es:

$$T^{(i)} = \min\left\{ t \;\middle|\; \sum_{\tau=1}^{t} \delta d_\tau^{(i)} \geq d \right\} \tag{20}$$

Repitiendo (17)–(20) para $i = 1, \ldots, N$ se obtiene la distribución empírica $\{T^{(1)}, \ldots, T^{(N)}\}$

de la cual se extraen los percentiles:

$$P_q = Q_q\Big(\{T^{(i)}\}_{i=1}^{N}\Big), \quad q \in \{10, 50, 90\} \tag{21}$$

# 7 Implementación del Motor Monte Carlo

## 7.1 Clase `MonteCarloEngine`

El módulo `src/simulation/monte_carlo.py` implementa el motor con simulación completamente **vectorizada** sobre el eje de trayectorias ($N \times T$ operaciones simultáneas via NumPy), eliminando cualquier bucle de Python sobre las simulaciones.

**Algoritmo 1:** Motor Monte Carlo vectorizado de UrbanFlow CDMX

**Input:** Cadena ajustada $\hat{\mathbf{P}}$, distancia $d$ km, estado inicial $s_0$, $N = 10{,}000$, $T_{\max} = 480$ pasos

**Output:** $P_{10}$, $P_{50}$, $P_{90}$, $\bar{T}$, $\sigma_T$

Precomputar $\mathbf{P}_{\text{cum}} \leftarrow \texttt{cumsum}(\hat{\mathbf{P}},\ \text{axis} = 1)$;

`// Paso 1: Generar trayectorias de estados (vectorizado)`

$\mathbf{S} \leftarrow \mathbf{0}_{N \times T_{\max}}$ ; `// matriz de estados int8`

$\mathbf{S}[:, 0] \leftarrow s_0$;

**for** $t = 1$ **to** $T_{max} - 1$ **do**

  $\mathbf{u} \sim \mathcal{U}[0, 1)^N$ ; `// N números uniformes simultáneos`

  $\mathbf{S}[:, t] \leftarrow \Big(\mathbf{u}_{:,\text{new}} \geq \mathbf{P}_{\text{cum}}[\mathbf{S}[:, t-1]]\Big).\text{sum}(\text{axis} = 1)$;

**end**

`// Paso 2: Muestrear velocidades por estado (máscara booleana)`

$\mathbf{V} \leftarrow \mathbf{0}_{N \times T_{\max}}$;

**for** $s \in \{0, 1, 2\}$ **do**

  $\text{mask} \leftarrow (\mathbf{S} = s)$;

  $\mathbf{V}[\text{mask}] \sim \mathcal{TN}(\mu_s, \sigma_s; [v_s^{\min}, v_s^{\max}])$;

**end**

`// Paso 3: Distancia acumulada e interpolación sub-paso`

$\mathbf{D} \leftarrow \texttt{cumsum}(\mathbf{V} \cdot \frac{\Delta t}{60},\ \text{axis} = 1)$;

$\text{idx} \leftarrow \texttt{argmax}(\mathbf{D} \geq d,\ \text{axis} = 1)$;

$\mathbf{T} \leftarrow \Big(\text{idx} + \text{fracción lineal}\Big) \cdot \Delta t$;

**return** $P_{10}, P_{50}, P_{90} \leftarrow \texttt{percentile}(\mathbf{T},\ [10, 50, 90])$;

## 7.2 Generación vectorizada de estados de Markov

El paso más costoso es la generación de $N \times T_{\max}$ estados. En lugar de llamar a `rng.choice()` $N$ veces en bucle, se usa el **método de la transformada inversa vectorizado**:

Listing 4: Generación de $N$ estados simultáneos por muestreo de CDF acumulada.

```python
def _simular_estados(self, estado_inicial: int) -> np.ndarray:
    n, T = self.n_simulaciones, self.max_pasos
    estados = np.empty((n, T), dtype=np.int8)
    estados[:, 0] = estado_inicial

    for t in range(1, T):
        actual = estados[:, t - 1]              # (N,) estado actual por simulacion
        u = self._rng.random(n)                 # (N,) numeros uniformes [0, 1)

        # P_cumsum[actual] -> (N, 3): CDF acumulada por simulacion
        # Contar cuantos umbrales son < u determina el siguiente estado
        siguiente = (u[:, np.newaxis] >= self._P_cumsum[actual]).sum(axis=1)
        estados[:, t] = np.clip(siguiente, 0, N_ESTADOS - 1).astype(np.int8)

    return estados  # shape: (N, T_max)
```

*Ejemplo del método de transformada inversa*

Para el estado **Lento** ($i = 1$), la CDF acumulada de la fila 1 de $\hat{\mathbf{P}}$ es:

$$\mathbf{P}_{\text{cum}}[1] = [0.18,\ 0.79,\ 1.00]$$

Dado $u \sim \mathcal{U}[0, 1)$, el siguiente estado se asigna según:

$$j^* = \sum_{j=0}^{2} \mathbf{1}[u \geq \mathbf{P}_{\text{cum}}[1, j]]$$

| $u$ | $u \geq 0.18$ | $u \geq 0.79$ | $j^* =$ Estado siguiente |
|---|---|---|---|
| 0.10 | | | 0 (Fluido) |
| 0.45 | ✓ | | 1 (Lento) |
| 0.85 | ✓ | ✓ | 2 (Congestionado) |

Tabla 6: Método de transformada inversa para muestreo del estado siguiente. ✓ = condición cumplida; = condición no cumplida.

### 7.3 Cálculo del tiempo de llegada con interpolación lineal

Para evitar el redondeo al minuto más cercano, se aplica interpolación lineal sub-paso. Sea $t^* = \texttt{argmax}_t(D_t^{(i)} \geq d)$ el primer paso donde se supera la distancia objetivo:

$$T^{(i)} = \left( t^* - 1 + \frac{d - D_{t^*-1}^{(i)}}{D_{t^*}^{(i)} - D_{t^*-1}^{(i)}} \right) \cdot \Delta t \quad \text{(minutos)} \tag{22}$$

Esta expresión da precisión sub-minuto sin necesidad de reducir $\Delta t$.

### 7.4 Estructura del resultado

Listing 5: Dataclass `ResultadoSimulacion`: salida completa del motor.

```
@dataclass
class ResultadoSimulacion:
    tiempos_minutos: np.ndarray  # distribucion completa (N = 10,000 valores)
    p10:  float                  # percentil 10: viaje rapido
    p50:  float                  # percentil 50: mediana (mas probable)
    p90:  float                  # percentil 90: viaje pesimista
    media: float                 # tiempo esperado E[T]
    std:   float                 # desviacion estandar sigma_T
    banda_incertidumbre: float   # P90 - P10 (amplitud de incertidumbre)
    n_recortadas: int            # trayectorias que excedieron max_pasos
```

**Ejemplo de resultado real**

Para un recorrido de 12.5 km con estado inicial **Lento** y lluvia leve ($\phi = 0.88$):

$$P_{10} = 28.3 \text{ min}, \quad P_{50} = 37.1 \text{ min}, \quad P_{90} = 52.8 \text{ min}, \quad \text{Banda} = 24.5 \text{ min}$$

# 8 Pipeline Integrador en Tiempo Real

## 8.1 Inferencia del estado de tráfico

El estado inicial $s_0$ de la cadena de Markov se infiere del ratio de congestión promedio del corredor $\bar{r}$ (ecuación (1)):

$$s_0 = \begin{cases} 0 \text{ (Fluido)} & \text{si } \bar{r} \geq 0.75 \\ 1 \text{ (Lento)} & \text{si } 0.45 \leq \bar{r} < 0.75 \\ 2 \text{ (Congestionado)} & \text{si } \bar{r} < 0.45 \end{cases} \tag{23}$$

Los umbrales $\{0.75, 0.45\}$ están calibrados empíricamente para la ZMVM y son consistentes con la clasificación del TomTom Traffic Index.

## 8.2 Ajuste climático de los parámetros de velocidad

El factor climático $\phi$ modifica las distribuciones de velocidad antes de iniciar la simulación Monte Carlo:

$$\mu'_s = \phi \cdot \mu_s, \quad v_s'^{\min} = \phi \cdot v_s^{\min}, \quad v_s'^{\max} = \phi \cdot v_s^{\max} \tag{24}$$

La desviación estándar $\sigma_s$ no se modifica: la dispersión del tráfico no cambia por la lluvia, sólo disminuye la velocidad media.

## 8.3 Flujo completo de predicción

1. **TomTom Routing API** $\rightarrow$ $d$ km, waypoints (geometría del mapa)
2. **TomTom Traffic Stats** $\rightarrow$ $\{r_k\}_{k=1}^{K}$, $\bar{r}$ promedio
3. Inferir $s_0$ con (23)
4. **OWM API** $\rightarrow$ condición climática $\rightarrow$ factor $\phi$
5. Ajustar parámetros de velocidad con (24)
6. `ConsultaViaje`$(d, s_0)$ $\rightarrow$ `MonteCarloEngine.correr()`
7. $\Rightarrow$ $P_{10}$, $P_{50}$, $P_{90}$ en $\approx$ 500 ms

# 9 Resultados y Validación

## 9.1 Convergencia del estimador Monte Carlo

A partir del error estándar del estimador (15) y de los rangos de tiempo típicos de la ZMVM ($\sigma_T \approx$ 15–20 min):

| $N$ | Error est. $P_{50}$ (min) | Error est. $P_{90}$ (min) |
|---|---|---|
| 100 | $\pm 5.2$ | $\pm 8.1$ |
| 1 000 | $\pm 1.6$ | $\pm 2.6$ |
| 10 000 | $\pm 0.5$ | $\pm 0.8$ |
| 100 000 | $\pm 0.2$ | $\pm 0.3$ |

Tabla 7: Error de estimación de percentiles en función de $N$. Balance óptimo con $N = 10{,}000$: precisión $\pm 0.5$ min, cómputo $< 1$ s.

### 9.2 Bandas de incertidumbre por estado de tráfico

Para un corredor de 15 km bajo distintas condiciones iniciales:

| **Estado inicial** | $P_{10}$ (min) | $P_{50}$ (min) | $P_{90}$ (min) | Banda (min) |
|---|---|---|---|---|
| **Fluido** | 22 | 28 | 38 | 16 |
| **Lento** | 30 | 42 | 61 | 31 |
| **Congestionado** | 48 | 72 | 105 | 57 |

Tabla 8: Bandas de incertidumbre $P_{10}/P_{50}/P_{90}$ para un recorrido de 15 km.

La banda $P_{90} - P_{10}$ se amplía con la congestión, capturando la mayor variabilidad del tráfico denso: cuando el tráfico está fluido la banda es de 16 min, pero en congestión alcanza 57 min.

### 9.3 Métrica objetivo MAPE

La métrica de éxito del proyecto es:

$$\text{MAPE} = \frac{1}{n} \sum_{k=1}^{n} \frac{|T_k^{\text{real}} - T_k^{P_{50}}|}{T_k^{\text{real}}} \times 100\,\% < 15\,\% \tag{25}$$

donde $T_k^{P_{50}}$ es la mediana de la distribución Monte Carlo para la consulta $k$. El umbral del 15 % fue establecido como objetivo del diplomado para horizontes de predicción de 30 minutos.

## 10 Conclusiones y Trabajo Futuro

### 10.1 Conclusiones

1. **La modelación estocástica supera al enfoque determinístico.** Las bandas de incertidumbre $P_{10}/P_{90}$ proveen información accionable que un tiempo puntual no puede ofrecer: el usuario puede planearse con el nivel de certeza que necesite.

2. **Las cadenas de Markov son un modelo adecuado para la dinámica del tráfico.** La propiedad de Markov (3) es una aproximación razonable: el estado vial en $t+1$ está correlacionado principalmente con el estado en $t$. Los elementos diagonales dominantes en $\hat{\mathbf{P}}$ confirman la inercia del tráfico observada en la ZMVM.

3. **Monte Carlo con $N = 10{,}000$ trayectorias es suficiente.** El error estándar del estimador de $P_{50}$ es $\pm 0.5$ min, con tiempo de cómputo menor a 1 segundo en CPU sin GPU.

4. **El factor climático mejora la calibración.** El ajuste multiplicativo (24) captura el efecto de la lluvia sobre las distribuciones de velocidad de forma simple y sin sobreajuste.

5. **La TomTom Routing API es una mejora sustancial.** La red vial urbana tiene tortuosidad media del 40 % (factor $\kappa = 1.4$); usar la distancia real por carretera reduce el sesgo de la predicción.

### 10.2 Trabajo futuro

## Referencias


1. Norris, J. R. (1997). *Markov Chains*. Cambridge University Press.
2. Rubinstein, R. Y., & Kroese, D. P. (2016). *Simulation and the Monte Carlo Method* (3rd ed.). Wiley.
3. TomTom Traffic Index 2024 — Ciudad de México. `https://www.tomtom.com/traffic-index/`
4. SEMOVI (2023). *Aforos vehiculares en la Zona Metropolitana del Valle de México*. Secretaría de Movilidad, Gobierno de la Ciudad de México.
5. Harris, C. R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. `https://doi.org/10.1038/s41586-020-2649-2`
6. Van Lint, J. W. C., & Hoogendoorn, S. P. (2010). A Robust and Efficient Method for Fusing Heterogeneous Data from Traffic Sensors. *Computer-Aided Civil and Infrastructure Engineering*, 25(8), 596–612.
7. Sinnott, R. W. (1984). Virtues of the Haversine. *Sky and Telescope*, 68(2), 159.
8. Streamlit Documentation (2024). *Streamlit — The fastest way to build data apps.* `https://docs.streamlit.io`


## A Parámetros de Configuración del Motor

## B Glosario

**Cadena de Markov** Proceso estocástico donde el estado futuro depende únicamente del estado presente (propiedad de Markov).

**Distribución estacionaria ($\boldsymbol{\pi}^*$)** Vector de probabilidades al que converge la cadena de Markov en el largo plazo.

**Monte Carlo** Método de estimación basado en muestreo aleatorio masivo.

**Normal truncada** Distribución normal restringida a un intervalo $[v^{\min}, v^{\max}]$.

$P_{10}/P_{50}/P_{90}$ Percentiles 10, 50 y 90 de la distribución simulada de tiempos de viaje.

**Ratio de congestión ($r$)** Cociente $v_{\text{actual}}/v_{\text{libre}}$; mide el grado de congestión del segmento vial.

**MAPE** *Mean Absolute Percentage Error*; error porcentual absoluto medio.

**ZMVM** Zona Metropolitana del Valle de México.

**WGS84** Sistema de coordenadas geográficas estándar (GPS, EPSG:4326).

**EPSG:6372** Sistema de referencia cartográfico oficial para México (para cálculos de distancia en metros).

| **Prioridad** | **Mejora** | **Impacto esperado** |
|---|---|---|
| Alta | Matrices de Markov diferenciadas por hora y corredor | Capturar patrón hora-pico vs. hora-valle |
| Alta | Pipeline de entrenamiento con datos C5 CDMX (+1M registros) | Estimaciones $\hat{p}_{ij}$ estadísticamente robustas |
| Media | API REST FastAPI + caché Redis | Reducir latencia a < 100 ms |
| Media | Integración con Metrobús GTFS-RT | Predicción multimodal (auto + transporte público) |
| Baja | Motor Monte Carlo en GPU (CuPy) | Reducir tiempo de |

| **Parámetro** | **Valor por defecto** | **Descripción** |
|---|---|---|
| `n_simulaciones` | 10,000 | Número de trayectorias Monte Carlo |
| `paso_minutos` | 1.0 | Resolución temporal $\Delta t$ (min) |
| `max_pasos` | 480 | Horizonte máximo (8 horas) |
| `suavizado` | $10^{-6}$ | Constante de Laplace $\varepsilon$ |
| `FACTOR_TORTUOSIDAD` | 1.4 | Factor $\kappa$ Haversine $\rightarrow$ carretera |
| `UMBRAL_FLUIDO` | 0.75 | Umbral de ratio para estado Fluido |
| `UMBRAL_LENTO` | 0.45 | Umbral de ratio para estado Lento |

Tabla 10: Parámetros configurables del motor estocástico.